QUICK SORT

- It uses the idea of divide and conquer approach.
- Quick Sort follows a recursive algorithm.
- It divides the given array into two sections using a partitioning element called as pivot.

The division performed is such that-

- All the elements to the left side of pivot are smaller than pivot.
- All the elements to the right side of pivot are greater than pivot.
- After dividing the array into two sections, the pivot is set at its correct position.
- Then, sub arrays are sorted separately by applying quick sort algorithm recursively.
- It is based on the splitting of an array (partition) into smaller ones and swapping (exchange) based on the comparison with 'pivot' element selected.
- Due to this, quick sort is also called as "Partition Exchange" sort.

Technically, quick sort follows the below steps:

Step 1 – Make any element as pivot

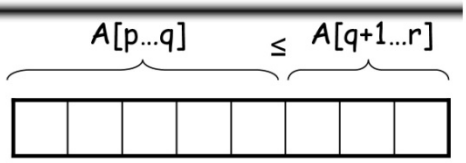
Step 2 – Partition the array on the basis of pivot

Step 3 – Apply quick sort on left partition recursively

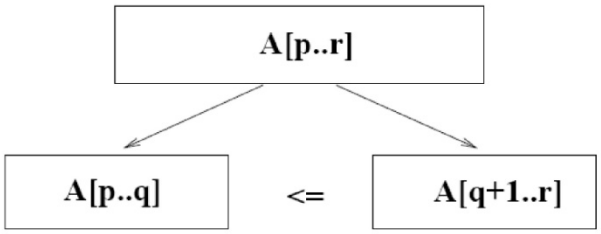
Step 4 – Apply quick sort on right partition recursively

- Quick sort works by partitioning a given array A[p … r] into two non-empty sub array
- A[p … q] and A[q+1 … r] such that every key in A[p … q] is less than or equal to every key in A[q+1 … r].
- Then, the two sub-arrays are sorted by recursive calls to Quick sort.
- The exact position of the partition depends on the given array and index q is computed as a part of the partitioning procedure.

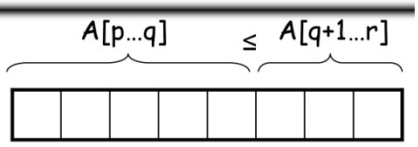
Quicksort



- Sort an array $A[p...r]$
- **Divide**
  - Partition the array A into 2 subarrays $A[p..q]$ and $A[q+1..r]$, such that each element of $A[p..q]$ is smaller than or equal to each element in $A[q+1..r]$
  - Need to find index q to partition the array



Quicksort



- **Conquer**
  - Recursively sort $A[p..q]$ and $A[q+1..r]$ using Quicksort
- **Combine**
  - Trivial: the arrays are sorted in place
  - No additional work is required to combine them
  - The entire array is now sorted

Algorithm: Quick-Sort (A, p, r)

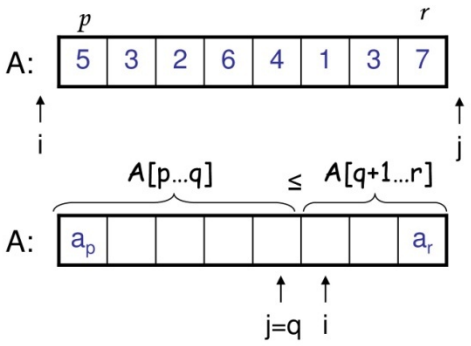
```
if p < r then
   q= Partition (A, p, r)
   Quick-Sort (A, p, q-1)
   Quick-Sort (A, q + 1, r)
```

Initially p=1, r=n

Partitioning the Array

Alg. PARTITION (A, p, r)

```
1.  x ← A[p]
2.  i ← p - 1
3.  j ← r + 1
4.  while TRUE
5.        do repeat j ← j - 1
6.           until A[j] ≤ x
7.        do  repeat i ← i + 1
8.           until A[i] ≥ x
9.        if i < j
10.          then exchange A[i] ↔ A[j]
11.       else return j
```



Each element is visited once!

Running time: $\Theta(n)$
$n = r - p + 1$